



**INNOPOLIS
UNIVERSITY**

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 5 · Пространственная фильтрация

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография: оценивание и выравнивание	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	L12 Стерео и эппиплярная геометрия	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — **L5: Пространственная фильтрация**

RECAP

- **Ресемплинг не создаёт информацию:** Увеличение разрешения (апскейлинг) не добавляет скрытых деталей. Как бы вы ни увеличивали размытый текст, его качество (PSNR) останется низким. При выпрямлении перспективы пиксели могут совпасть идеально, но оценка реальных физических размеров «на глаз» по краям кадра дает огромную погрешность (27%).
- **Размытие перед уменьшением необходимо:** Перед тем как сжать картинку (downsampling/warp), ее нужно слегка размыть по Гауссу ($\sigma = 1.5$). Это убирает визуальные артефакты (алиасинг) и драматически повышает качество результата — в примере PSNR вырос с 27.9 до 36.7 дБ.
- **Метрика PSNR слепа к геометрии:** PSNR сравнивает строго пиксель-в-пиксель. Если сдвинуть идеальную картинку всего на 1 пиксель, PSNR покажет плохой результат (26 дБ), хотя визуально картинка не изменилась. Поэтому для оценки структуры вводят метрику SSIM.

Самопроверка L4

Трансформации: Для аффинного преобразования нужно минимум 3 пары точек, для гомографии — 4 пары.

Поворот: При повороте изображения на 30° холст под него сильно разрастается, из-за чего 48% пространства занимают пустые черные области.

Вид сверху (Bird's-eye view): При проекции на расстоянии 20 метров плотность падает до 5 строк на метр. Увеличение фокусного расстояния f_x в 2 раза увеличивает количество строк тоже в 2 раза (линейно), а не в 4.

LEARNING OBJECTIVES

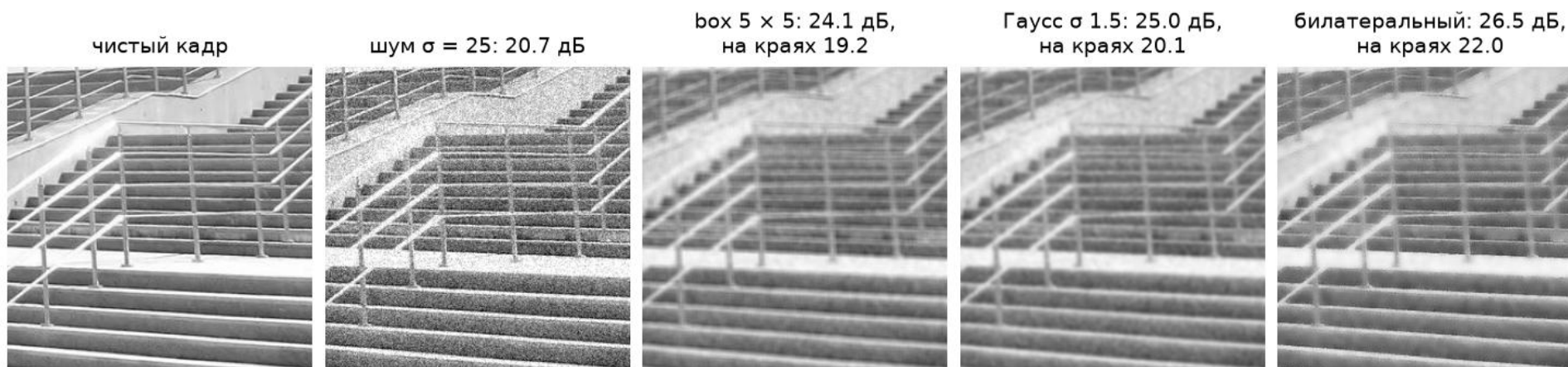
- **Записывать** линейный фильтр как свёртку с ядром, отличить её от корреляции и назвать четыре свойства LSI-фильтра с последствиями для кода
- **Выбирать** сглаживание под модель шума: box, Гаусс, медиана, билатеральный — и обосновать выбор числом PSNR на краях и на ровных областях
- **Оценить** цену фильтра: k^2 против $2k$ умножений, сепарабельность, интегральное изображение, миллисекунды на 1920×1279
- **Применять** `filter2D`, `GaussianBlur`, `medianBlur`, `bilateralFilter`, `matchTemplate` и объяснить каждый параметр, включая `ddepth` и границу
- **Распознавать** три молчаливых отказа: переполнение `uint8`, чётное ядро, корреляция без нормировки

Убрать шум, не убив края

МОТИВАЦИЯ

Дано: кадр в сумерках, шум $\sigma = 25$ (20.7 дБ). **Задача:** Как убрать цифровой шум на изображении, не размыв при этом важные контуры и границы объектов.

- **Размытие в лоб не работает:** Простой усредняющий фильтр (box 5×5) и фильтр Гаусса ($\sigma 1.5$) немного поднимает PSNR до 24.1 дБ, но **на краях** и контурах они делают картинку хуже, чем она была до фильтрации (качество падает до 19.2 дБ и 20.1 дБ соответственно). Линии просто «размываются».
- **Решение – билатеральный фильтр.** Этот алгоритм сглаживает шум, но пасует перед резкими перепадами цвета (границами). Он показал лучший результат как для всего кадра (26.5 дБ), так и для краев (22.0 дБ).
- **Робототехника:** Sanny по шумному кадру даёт 874 тыс. «краёв» вместо 247 тыс.; фильтр — первый шаг любого детектора · **МОИИ:** предобработка, аугментации, первый слой CNN



Один кроп лестницы: шум, три фильтра, два числа у каждого — весь кадр и только края

© фото — здание Университета Иннополис; собственная иллюстрация

1

Линейная фильтрация: свёртка

- Окно, ядро, корреляция и свёртка · границы · box и Гаусс · сепарабельность и интегральные изображения · шум и резкость

- Monochrome image $\Rightarrow$ matrix of intensity values
- Typical sizes:
 - 320 x 240 (QVGA)
 - 640 x 480 (VGA)
 - 1280 x 720 (HD)

Соотношение сторон 4:3

Соотношение сторон 16:9
- Intensities sampled to 256 grey levels $\Rightarrow$ 8 bits
- Images capture a lot of information
 - 1920 x 1080 (FullHD, 1080p)
 - 2560 x 1440 (2K, QHD, 1440p)
 - 3840 x 2160 (4K, UHD, 2160p)

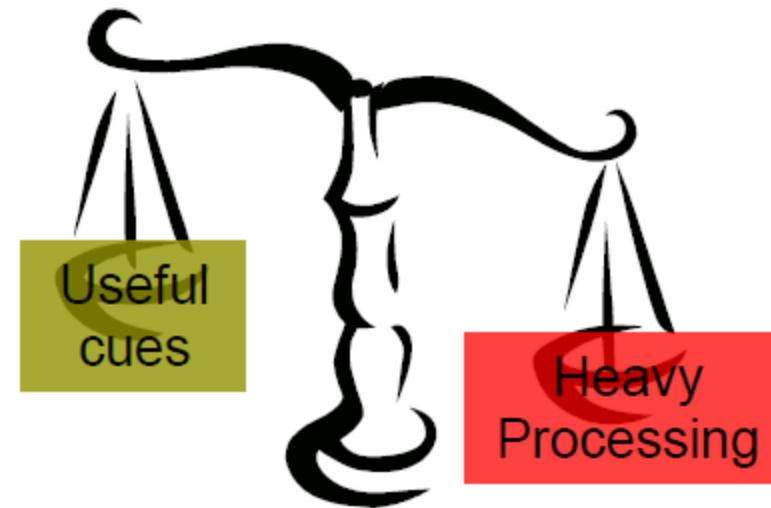


Image Intensities & Data Reduction



218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218
218 218 218 218 218 218 218 218 218 218 218 218

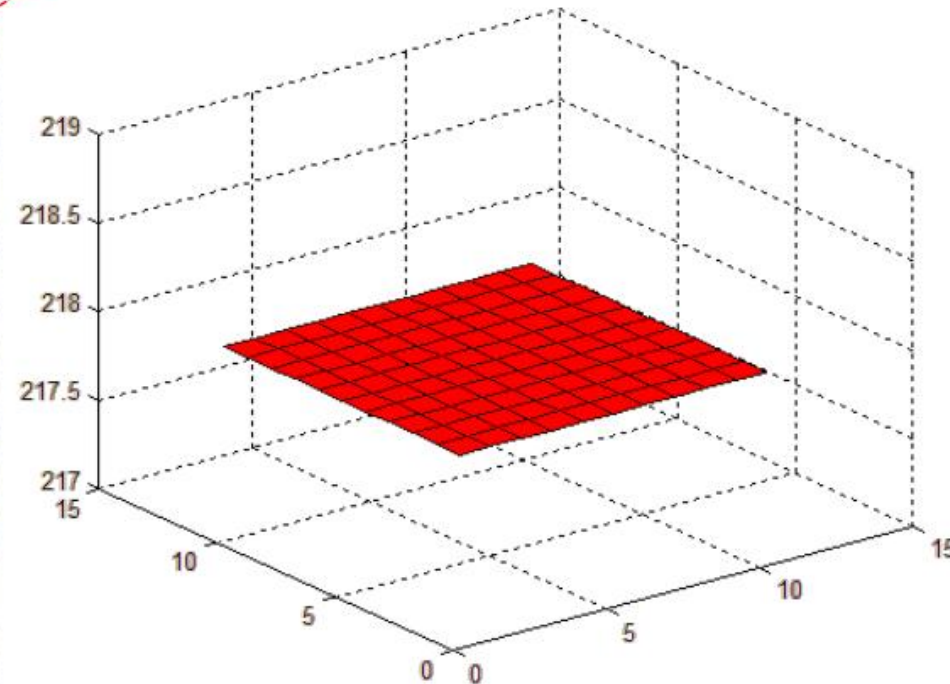
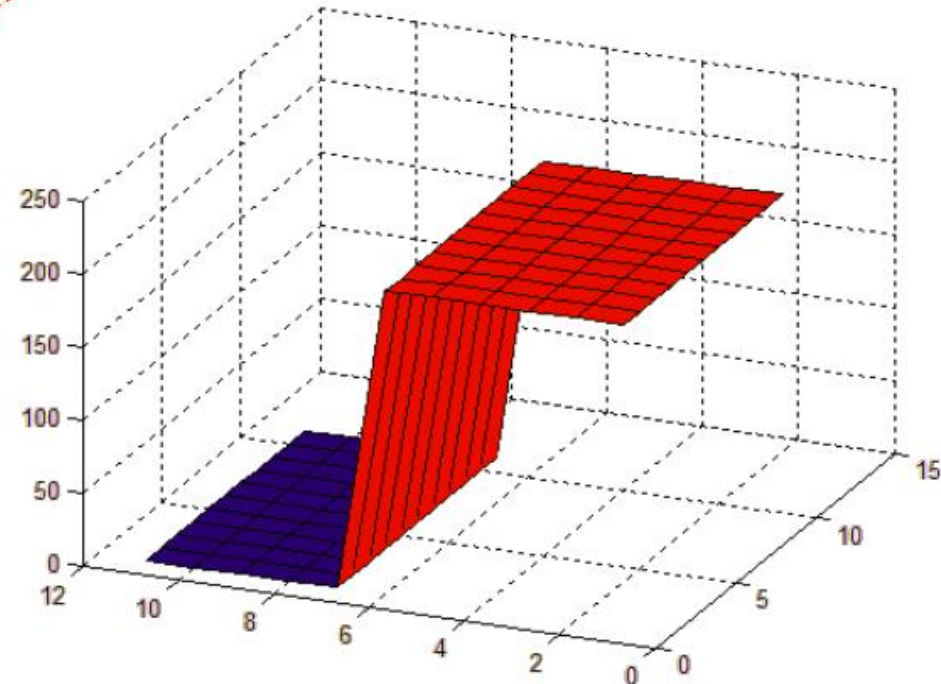


Image Intensities & Data Reduction



208	208	208	208	208	208	208	208	208	208	208	208
208	208	208	208	208	208	208	208	208	208	208	208
208	208	208	208	208	208	208	208	208	208	208	208
208	208	208	208	208	208	208	208	208	208	208	208
208	208	208	208	208	208	208	208	208	208	208	208
208	207	208	208	208	208	208	208	208	208	208	208
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0





```
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 230 229 229 229 229 229
5 17 31 7 1 0 229 229 229 229 229
0 0 1 0 0 0 229 229 229 229 229
0 0 0 0 0 0 229 229 229 229 229
0 0 0 0 1 4 229 229 229 229 229
0 0 0 0 0 11 229 229 229 229 229
0 0 0 0 0 5 229 229 229 229 229
```

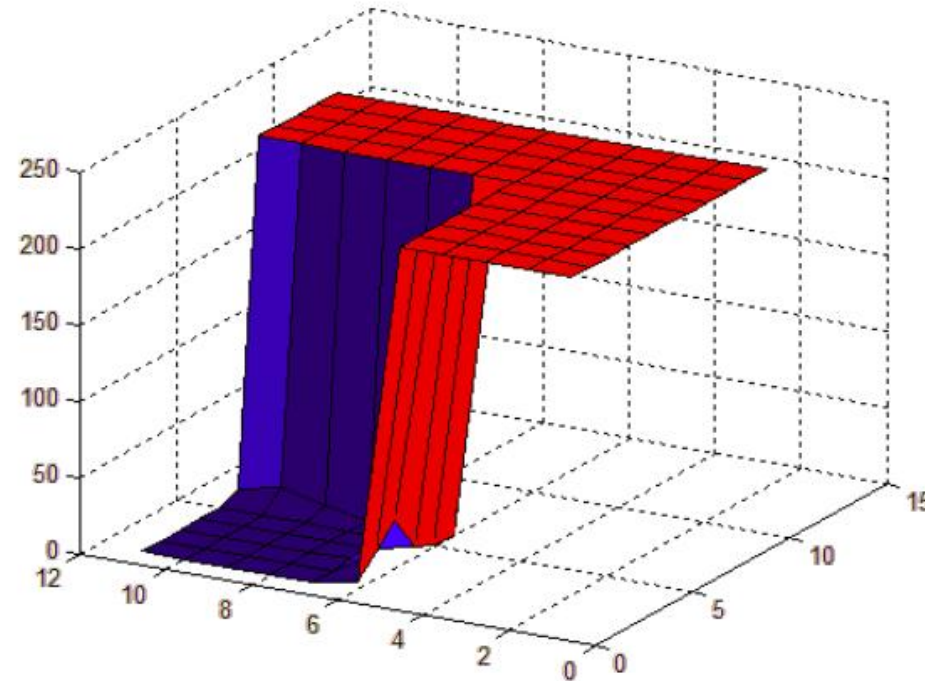


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html



```
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 229 229 229 229 229 229
229 229 229 229 229 230 229 229 229 229 229
5 17 31 7 1 0 229 229 229 229 229
0 0 1 0 0 0 229 229 229 229 229
0 0 0 0 0 0 229 229 229 229 229
0 0 0 0 1 4 229 229 229 229 229
0 0 0 0 0 11 229 229 229 229 229
0 0 0 0 0 5 229 229 229 229 229
```

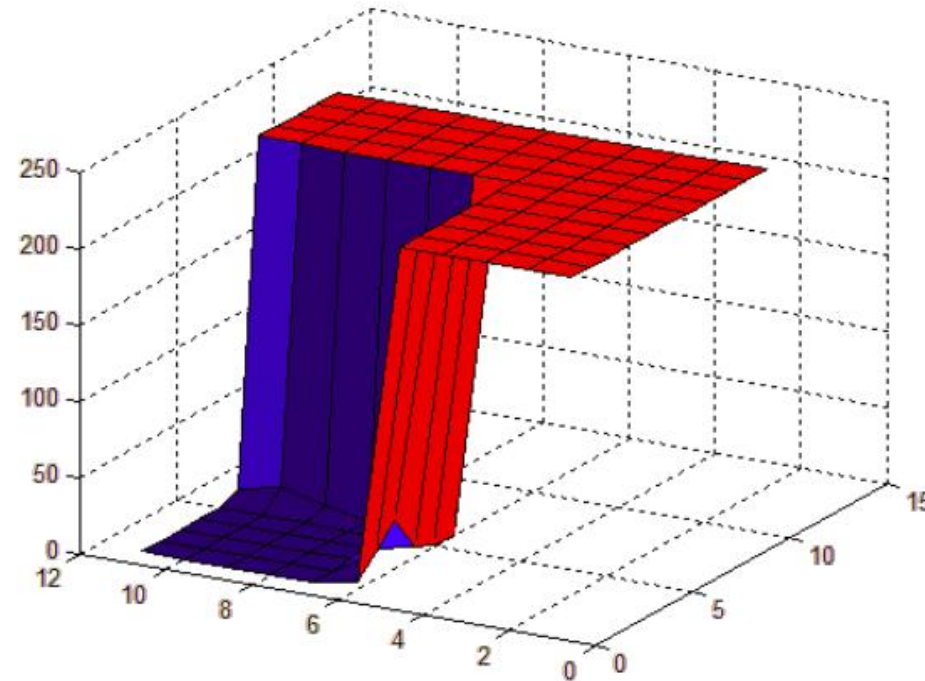
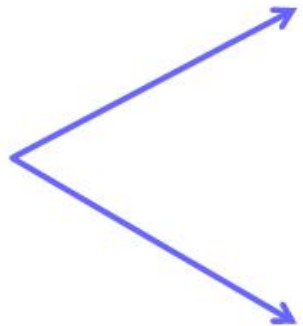


Fig. © Margarita Chli, Paul Furgale, Marco Hutter, Martin Rufli, Davide Scaramuzza, Roland Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

- **filtering:** accept / reject certain components
- example: a low-pass filter allows low frequencies $\Rightarrow$ blurring (smoothing) effect on an image – used to reduce image noise
- Smoothing can also be achieved by **spatial filters** instead of **frequency filters**.



Low-pass filtering: retains low-frequency components (smoothing)

Размывающая маска (blurred mask) имеет следующие особенности:

- Все значения в маске положительны;
- Сумма всех значений равна 1;
- Размытие уменьшает резкость контуров и границ объектов на изображении, делая их менее заметными;
- При увеличении маски эффект размытия становится более плавным



High-pass filtering: retains high-frequency components (edge detection)

Маска с производными (derivative mask) имеет следующие свойства:

- Маска имеет как положительные, так и отрицательные значения;
- Сумма всех значений в такой маске равна нулю;
- Увеличивает контрастность и выделение краев объектов на изображении, делая их более заметными;
- Чем больше размер маски, тем больше выделяется тонких деталей и краёв изображений, что увеличивает общее количество обнаруживаемых краев

- **Linear:** every pixel is replaced by a linear combination of its neighbours
- **Shift-invariant:** the same operation is performed on every point on the image

- Basic & very useful filtering operations:

- Correlation
- Convolution

$$I(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \cdot I(x+s, y+t) \quad \text{Корреляция (correlation)}$$

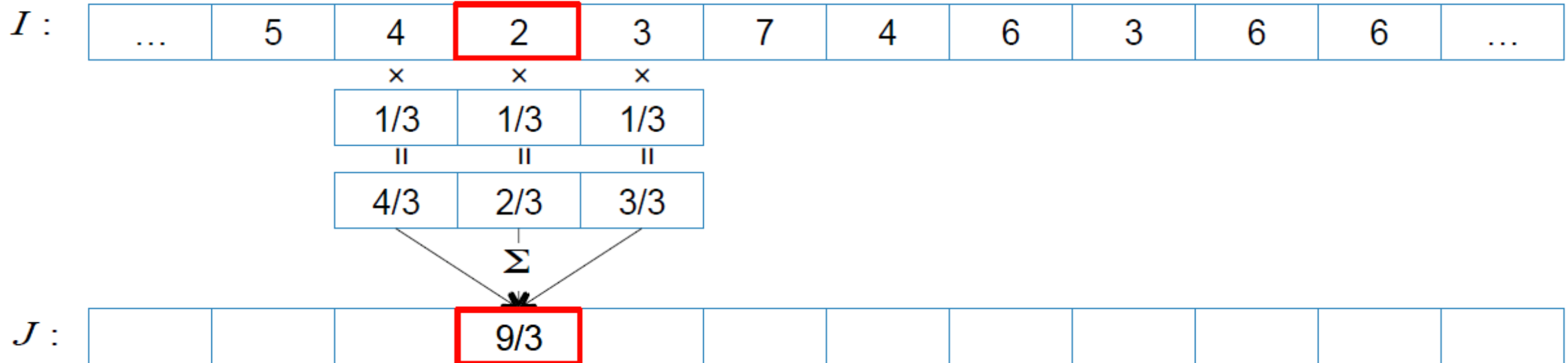
фильтр/ядро (kernel)

$$I(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \cdot I(x-s, y-t) \quad \text{Свертка (convolution)}$$

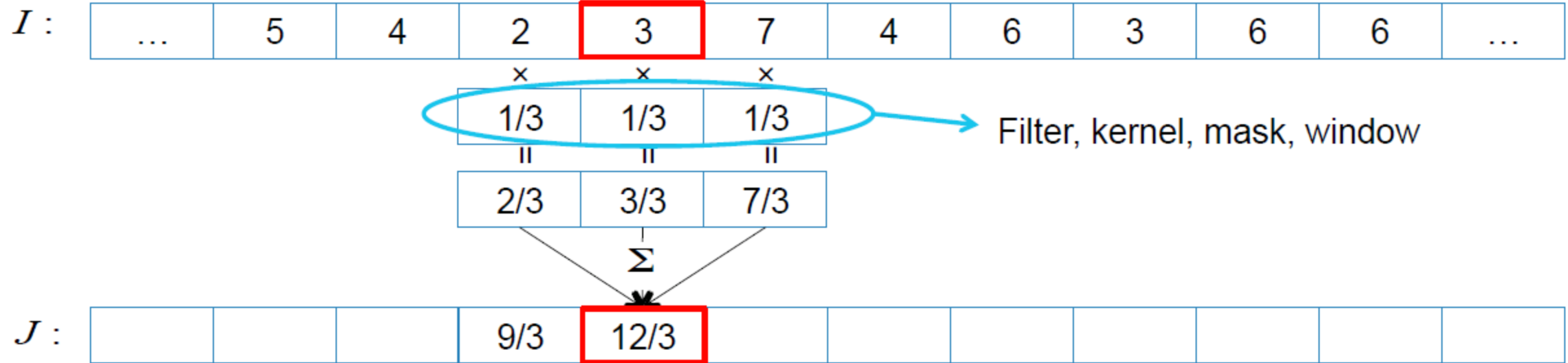
- Brief study of these filters in the simplest case of 1D images (i.e. a row of pixels) & their extension to 2D

- ❑ **Свёртка и корреляция:** Это два метода обработки изображений, часто используемые для фильтрации и обнаружения признаков на изображении.
- ❑ Свертка в математике и обработке сигнала определяется как интеграл от произведения двух функций, одна из которых переворачивается и сдвигается
- ❑ В **свёртке** используется знак минус, что требует **зеркального отражения фильтра** перед его применением к изображению. Это связано с тем, что свёртка сворачивает фильтр по всем осям, изменяя направление фильтра.
- ❑ Когда используется **симметричный фильтр** (неизменный при зеркальном отражении), **свёртка и корреляция** дают одинаковый результат, и являются взаимозаменяемыми.

- An averaging filter



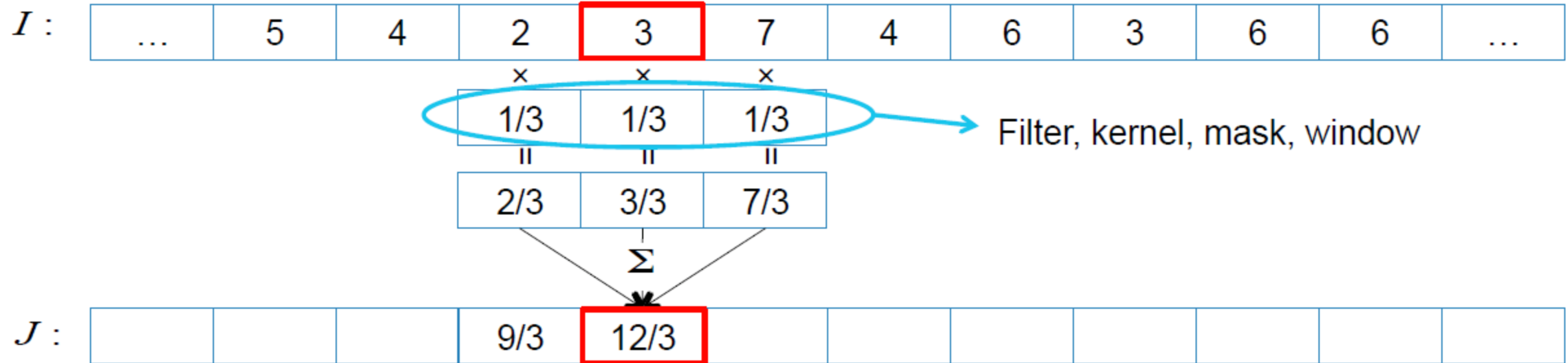
- An averaging filter



How to handle boundaries?

- Ignore filtered values at boundaries
- Pad image with zeros
- Pad image with first/last image values

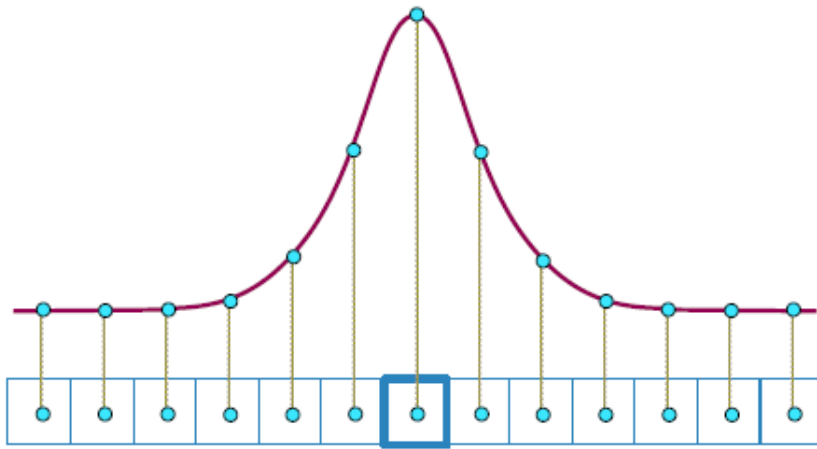
- An averaging filter



- Formally, Correlation is $J(x) = F \circ I(x) = \sum_{i \in [-N, N]} F(i)I(x+i)$

- In this smoothing example $F(i) = \begin{cases} 1/3, & i \in [-1, 1] \\ 0, & i \notin [-1, 1] \end{cases}$

- Common practice for image smoothing:
use a Gaussian



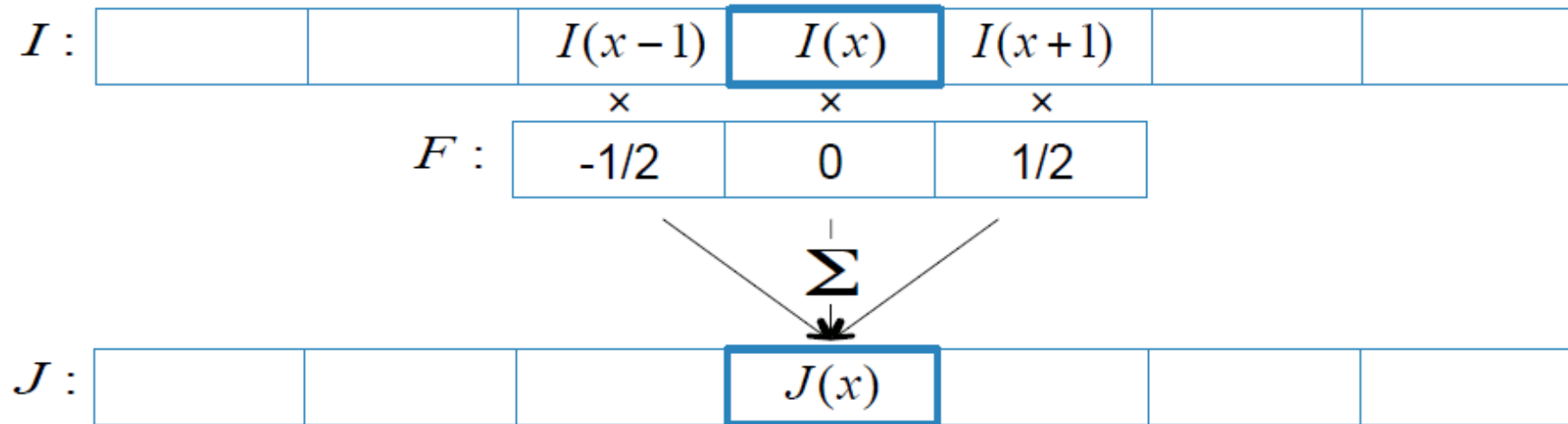
$$g(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$\mu = 0$$

σ : controls the amount of smoothing

- Near-by pixels have a bigger influence on the smoothed value rather than more distant ones

- **Derivative** of an image:
quantifies how quickly intensities change
(along the direction of the derivative)
- Approximate a derivative operator:



$$J(x) = \frac{I(x+1) - I(x-1)}{2}$$

Image filtering: matching using correlation

- Find locations in an image that are similar to a **template**
- Filter = template

3	8	3
---	---	---

 $\Rightarrow$ test it against all image locations

$I :$

3	2	4	1	3	8	4	0	3	8	7	7
---	---	---	---	---	---	---	---	---	---	---	---

- Similarity measure: Sum of Squared Differences (**SSD**) – minimize

$$\sum_{i=-N}^N (F(i) - I(x+i))^2$$

$J :$

26	37	21	50	54	1	50	65	59	16	42	17
----	----	----	----	----	---	----	----	----	----	----	----

Чем **меньше** итоговое число, тем **ближе** фрагмент к шаблону. Идеальное совпадение дает 0

Image filtering: matching using correlation

- Find locations in an image that are similar to a **template**
- Filter = template

3	8	3
---	---	---

 $\Rightarrow$ test it against all image locations

$I :$

3	2	4	1	3	8	4	0	3	8	7	7
---	---	---	---	---	---	---	---	---	---	---	---

- Similarity measure: Sum of Squared Differences (**SSD**) – minimize

$$\sum_{i=-N}^N (F(i) - I(x+i))^2 = \sum_{i=-N}^N (F(i))^2 + \sum_{i=-N}^N (I(x+i))^2 - 2 \sum_{i=-N}^N (F(i)I(x+i))$$

Энергия шаблона Энергия участка
картинки

Correlation

$J :$

26	37	21	50	54	1	50	65	59	16	42	17
----	----	----	----	----	---	----	----	----	----	----	----

Взаимная корреляция - попиксельное перемножение шаблона на картинку

Image filtering: constructing filter from a continuous fn

- Find locations in an image that are similar to a **template**
- Filter = template

3	8	3
---	---	---

 $\Rightarrow$ test it against all image locations

$I :$

3	2	4	1	3	8	4	0	3	8	7	7
---	---	---	---	---	---	---	---	---	---	---	---

- Similarity measure: Sum of Squared Differences (**SSD**) – minimize

$$\sum_{i=-N}^N (F(i) - I(x+i))^2 = \sum_{i=-N}^N (F(i))^2 + \sum_{i=-N}^N (I(x+i))^2 - 2 \sum_{i=-N}^N (F(i)I(x+i))$$

Correlation

$J :$

26	37	21	50	54	1	50	65	59	16	42	17
----	----	----	----	----	---	----	----	----	----	----	----

- Similarity measure: Correlation? – maximize

Ошибочно использовать максимум корреляции для схожести шаблона и картинки

$J :$

30	37	41	29	51	85	56	21	48	86	101	77
----	----	----	----	----	----	----	----	----	----	-----	----

Image filtering: Normalized Cross Correlation (NCC)

- Find locations in an image that are similar to a **template**
- Filter = template

3	8	3
---	---	---

 $\Rightarrow$ test it against all image locations

$I :$

3	2	4	1	3	8	4	0	3	8	7	7
---	---	---	---	---	---	---	---	---	---	---	---

- Correlation value is affected by the magnitude of intensities
- Similarity measure: Normalized Cross Correlation (**NCC**) – maximize

$$\frac{\sum_{i=-N}^{i=N} (F(i)I(x+i))}{\sqrt{\sum_{i=-N}^{i=N} (F(i))^2} \sqrt{\sum_{i=-N}^{i=N} (I(x+i))^2}}$$

Решение проблемы «ослепления»
корреляции яркими пятнами — метрика
Нормированной взаимной корреляции
(Normalized Cross Correlation, NCC)

$J :$

0.919	0.759	0.988	0.628	0.655	0.994	0.691	0.464	0.620	0.860	0.876	0.859
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Image filtering: Zero-mean NCC (ZNCC)

- Find locations in an image that are similar to a **template**
- Filter = template

3	8	3
---	---	---

 $\Rightarrow$ test it against all image locations

$I :$

3	2	4	1	3	8	4	0	3	8	7	7
---	---	---	---	---	---	---	---	---	---	---	---

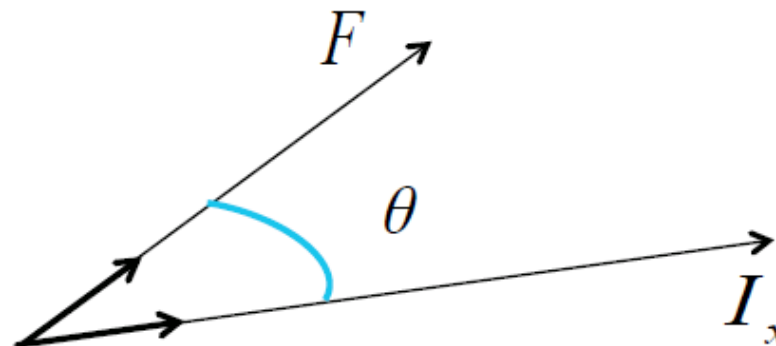
- Correlation value is affected by the magnitude of intensities
- Similarity measure: Zero-mean Normalized Cross Correlation (**ZNCC**) – maximize

$$\frac{\sum_{i=-N}^{i=N} (F(i) - \mu_F)(I(x+i) - \mu_{I_x})}{\sqrt{\sum_{i=-N}^{i=N} (F(i) - \mu_F)^2} \sqrt{\sum_{i=-N}^{i=N} (I(x+i) - \mu_{I_x})^2}}, \text{ where } \begin{cases} \mu_F = \frac{\sum_{i=-N}^N F(i)}{2N+1} \\ \mu_{I_x} = \frac{\sum_{i=-N}^N I(x+i)}{2N+1} \end{cases}$$

Более продвинутая метрика поиска шаблонов – ZNCC: полностью защищена от изменений освещенности и контраста

- Considering the filter F and the portion of the image I_x as vectors $\Rightarrow$ their correlation is:

$$\langle F, I_x \rangle = \|F\| \|I_x\| \cos \theta$$



Представим фильтр (шаблон) F и фрагмент изображения I_x не как набор пикселей, а как многомерные векторы. Корреляция — это скалярное произведение: поэлементное перемножение и сложения $\langle F, I_x \rangle$

- In **NCC** and **ZNCC** we consider the unit vectors of F and I_x , hence we measure their similarity based on the angle θ

Если мы отнормируем векторы, в формуле остаётся только $\cos \theta$, и метрики NCC и ZNCC оценивают сходство картинок исключительно по углу θ между их векторами

1. **Фильтры сглаживания** (размытие и уменьшение шума):
 - Эти фильтры применяются для **уменьшения шумов** (noise reduction) и **размытия изображения** (blurring), что помогает сделать изображение более гладким и менее шумным.
 - Размытие помогает удалять мелкие детали и заполнять небольшие пробелы, что полезно для сглаживания контуров и линий.
2. **Линейные и нелинейные фильтры**:
 - **Линейные фильтры** вычисляют средневзвешенное значение пикселей в маске фильтра, что приводит к сглаживанию изображения.
 - **Нелинейные фильтры**, такие как **медианные фильтры** выбирают медианное значение пикселей, чтобы удалить шум.
3. **Усредняющие фильтры (фильтры нижних частот)**:
 - Работают путем усреднения значений пикселей в определенной области, что приводит к сглаживанию изображения и уменьшению шума.
 - Они полезны для удаления высокочастотных шумов, таких как **"соль и перец"** (черные пиксели - перец и белые пиксели - соль), но они также могут размывать важные детали, такие как края.
4. **Размывание краев**:
 - Побочным эффектом уменьшения шума является размывание краев, что может ухудшить качество изображения.
 - Чтобы уменьшить этот эффект, можно выбрать соответствующие коэффициенты фильтра, которые сохраняют больше деталей на краях.
5. **Медианные фильтры**:
 - Это тип нелинейных фильтров, которые выбирают медианное значение пикселей в маске. Они особенно эффективны для удаления шумов типа "соль и перец", сохраняя при этом края и контуры изображений.

Предполагая использование маски 3x3, фильтр усреднения может быть записан как w , где сумма всех коэффициентов равна 1:

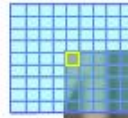
$$w = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- ❑ **Нормализация:** Нормализация нужна для того, чтобы итоговое значение не менялось, если все пиксели в области одинаковы. Это обеспечивает равномерное усреднение.
- ❑ **Коэффициенты фильтра:** Вместо того, чтобы использовать коэффициенты 1/9 для каждого элемента фильтра, используются коэффициенты, равные 1. Это упрощает процесс вычислений.
- ❑ **Суммирование и деление:** Все пиксели в маске фильтра суммируются, а затем делятся на 9 (размер маски 3x3), чтобы получить усредненное значение.

$$F \circ I(x, y) = \sum_{j \in [-M, M]} \sum_{i \in [-N, N]} F(i, j) I(x + i, y + j)$$

- Example:
Constant averaging filter

$$F = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$



This example was generated with a 21x21 mask



$$F \circ I(x, y) = \sum_{j \in [-M, M]} \sum_{i \in [-N, N]} F(i, j) I(x + i, y + j)$$

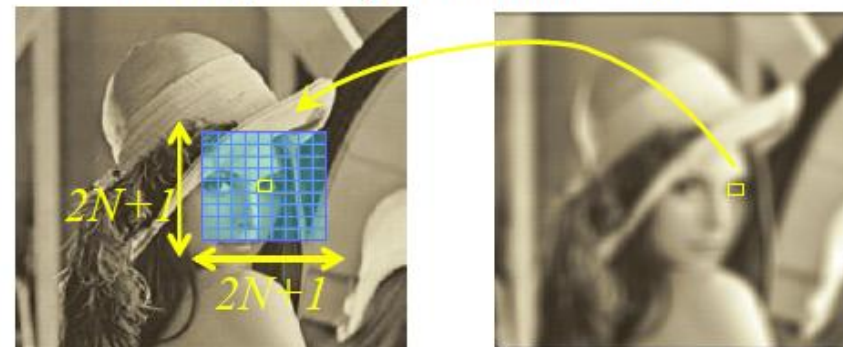
- Example:
Constant averaging filter

$$F = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

- If $\text{size}(F) = (2N + 1)^2$ i.e. this is a square filter

- 2D Correlation $\Rightarrow$ no. multiplications per pixel $= (2N + 1)^2$
no. additions per pixel $= (2N + 1)^2 - 1$

This example was generated with a 21x21 mask



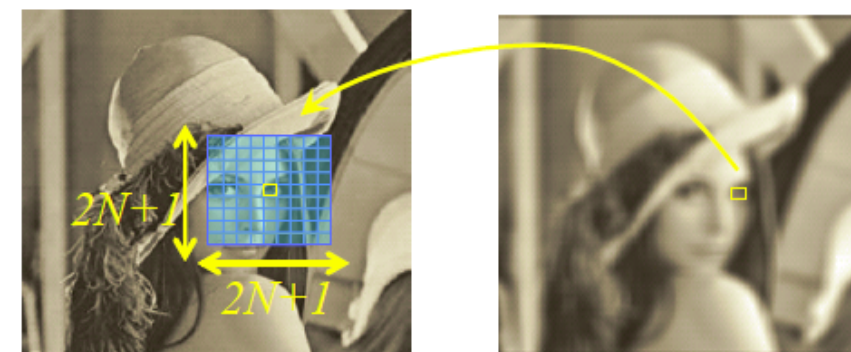
$$F \circ I(x, y) = \sum_{j \in [-M, M]} \sum_{i \in [-N, N]} F(i, j) I(x + i, y + j)$$

- Example:
Constant averaging filter

$$F = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix} = \begin{bmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{bmatrix} \cdot \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix} \quad \text{"separable" filter}$$

- If $\text{size}(F) = (2N + 1)^2$ i.e. this is a square filter
 - 2D Correlation $\Rightarrow$ no. multiplications per pixel $= (2N + 1)^2$
no. additions per pixel $= (2N + 1)^2 - 1$
 - $2 \times 1D$ Correlation $\Rightarrow$ no. multiplications per pixel $= 2(2N + 1)$
no. additions per pixel $= 4N$

This example was generated with a 21x21 mask



$$F \circ I(x, y) = \sum_{j \in [-M, M]} \sum_{i \in [-N, N]} F(i, j) I(x + i, y + j)$$

- Example:
Constant averaging filter

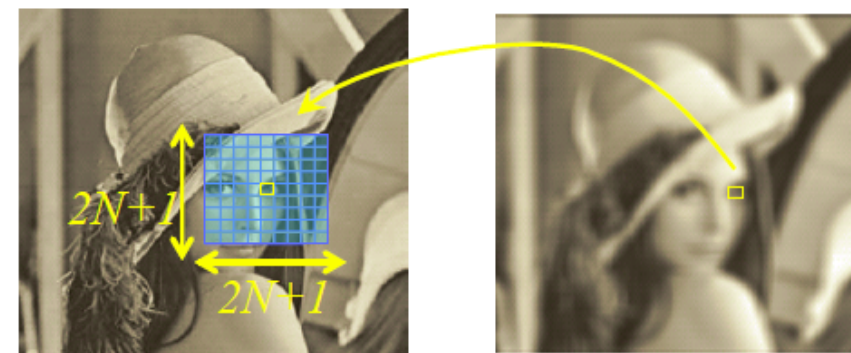
$$F = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix} = \begin{bmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{bmatrix} \cdot \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

“separable” filter

- If $\text{size}(F) = (2N + 1)^2$ i.e. this is a square filter

- 2D Correlation $\Rightarrow$ no. multiplications per pixel $= (2N + 1)^2$
no. additions per pixel $= (2N + 1)^2 - 1$
- 2×1 D Correlation $\Rightarrow$ no. multiplications per pixel $= 2(2N + 1)$
no. additions per pixel $= 4N$
- 2×1 D Correlation (with const. factor) $\Rightarrow$ no. multiplications per pixel $= 1$
no. additions per pixel $= 4N$

This example was generated with a 21x21 mask



1. Вторая производная интенсивности:

- Вторая производная интенсивности изображения (**лапласиан**) используется для выделения изменений в интенсивности, таких как края и контуры объектов. Это помогает улучшить четкость и детализацию изображения.

2. Чувствительность к изменениям освещенности:

- Алгоритмы, использующие вторую производную, очень чувствительны к изменениям освещенности, что может привести к увеличению шума и ошибкам в обнаружении особенностей. Поэтому важно сгладить сигнал перед применением таких алгоритмов.

3. Гауссовый усредняющий фильтр:

- Гауссовый фильтр используется для сглаживания изображения перед применением алгоритмов второй производной.
- Гауссовый фильтр имеет форму колоколообразной кривой, где центральные пиксели имеют больший вес, а пиксели на краях — меньший. Это позволяет плавно усреднять значения интенсивности пикселей.

$$G_{\sigma}(x, y) = e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

1. Создание фильтрующей маски 3x3:

- Фильтрующая маска 3x3 создается из гауссовской функции путем выборки значений функции вокруг её центра.
- Значение σ (сигма) определяет ширину гауссовского распределения. В данном случае $\sigma=0.85$.

2. Рескейлинг коэффициентов:

- Коэффициенты маски пересчитываются, чтобы их сумма равнялась 1. Это делается для нормализации маски и сохранения общей яркости изображения после применения фильтра.

$$G = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- Использование в маске коэффициентов, являющихся **степенями двойки** (например, 1, 2, 4, 8 и т.д.), значительно упрощает и ускоряет вычисления - вместо тяжелого деления и умножения использовать сверхбыструю побитовую **операцию сдвига** (bit shift).
- Такой фильтр, благодаря своей эффективности и простоте реализации, широко используется в приложениях обработки изображений.
- Такой **фильтр нижних частот** пропускает низкочастотные компоненты сигнала и подавляет высокочастотные компоненты, что приводит к сглаживанию изображения и устранению случайных высокочастотных шумов.
- Уменьшение высокочастотного шума делает первую и вторую производные интенсивности более стабильными.

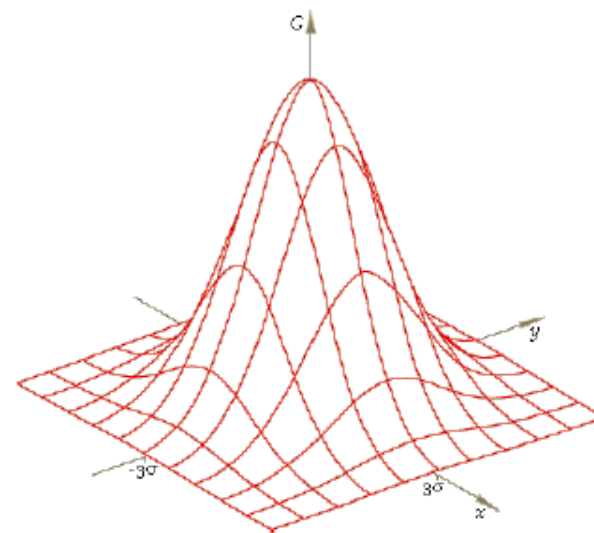
- A general, 2D Gaussian $G(x, y) = \frac{1}{2\pi|S|^{1/2}} e^{-\frac{1}{2}\begin{pmatrix} x \\ y \end{pmatrix}^T S^{-1} \begin{pmatrix} x \\ y \end{pmatrix}}$

Важное математическое и вычислительное свойство фильтра Гаусса — его **сепарабельность (разделяемость)**

- We usually want to smooth by the same amount in both x and y directions $S = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{bmatrix}$
- So this simplifies to:

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} = \underbrace{\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{x^2}{2\sigma^2}}}_{g_{\sigma}(x)} \cdot \underbrace{\frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{y^2}{2\sigma^2}}}_{g_{\sigma}(y)}$$

- Another separable filter



- Convolution is **equivalent** to Correlation with a flipped filter before correlating

- CONVOLUTION:** $J(x) = F * I(x) = \sum_{i \in [-N, N]} F(i) I(x - i)$

- CORRELATION:** $J(x) = F \circ I(x) = \sum_{i \in [-N, N]} F(i) I(x + i)$

- Likewise, in 2D we flip the filter both horizontally & vertically

$$J(x, y) = F * I(x, y) = \sum_{j \in [-M, M]} \sum_{i \in [-N, N]} F(i, j) I(x - i, y - j)$$

- Key difference between correlation and convolution is that **convolution is associative**:

$$F * (G * I) = (F * G) * I$$

- Very useful!
- Example: smooth an image & take its derivative $\Rightarrow$ convolve the Derivative filter with the Gaussian Filter & convolve the resulting filter with the Image

So if $F = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$
 $F' = \begin{bmatrix} 3 & 2 & 1 \end{bmatrix}$
Then, $F * I(x) = F' \circ I(x)$

Свертка — это та же корреляция, но с предварительно перевернутым (зеркально отраженным) фильтром

В отличие от корреляции, свёртка обладает свойством ассоциативности

Окно 3 × 3: одно скалярное произведение

окрестность 7 × 7 в кадре (значения пикселей)

93	97	101	103	99	97	97
97	91	97	104	108	109	108
36	34	32	30	32	44	60
158	144	140	130	118	111	106
247	248	253	255	255	255	255
251	246	251	254	253	251	254
250	248	252	254	250	243	235

ядро box 3 × 3

$$1/9 \cdot [1 \ 1 \ 1; 1 \ 1 \ 1; 1 \ 1 \ 1]$$

ядро Гаусса 3 × 3

$$1/16 \cdot [1 \ 2 \ 1; 2 \ 4 \ 2; 1 \ 2 \ 1]$$

$$g(x, y) = \sum h(i, j) \cdot I(x+i, y+j)$$

сумма по окну с весами ядра

один пиксель результата — одно скалярное произведение

центр окна: 130

$$\text{box: } (32 + 30 + 32 + 140 + 130 + 118 + 253 + 255 + 255) / 9 = 138.3$$

Гаусс: взвешенная сумма = 136.1

оба ядра суммируются в 1 — яркость равных областей не меняется; отличие — в весе центра: 1/9 против 4/16

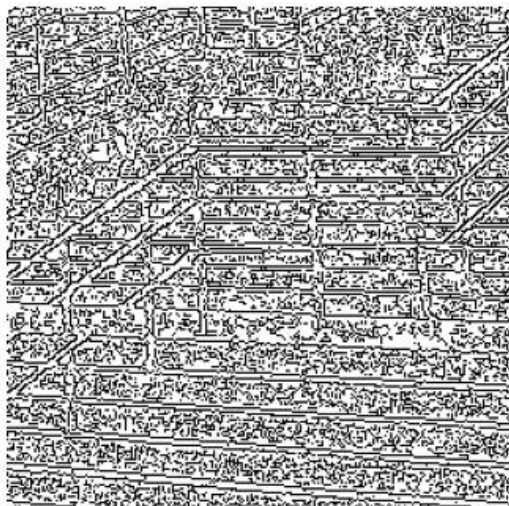
Когда фильтр скользит по изображению, для каждого пикселя окна (в окрестностях центрального, выделенного в оранжевой рамке) вычисляется **взвешенная сумма** его окрестности.

Окрестность 7 × 7 на краю ступени: значения пикселей, окно 3 × 3 вокруг центра 130 и два ядра. Box даёт 138.3, Гаусс — 136.1: оба ядра суммируются в единицу, поэтому на ровном месте яркость не меняется; отличие — в весе центра, 1/9 против 4/16

© фото — здание Университета Иннополис; собственная иллюстрация

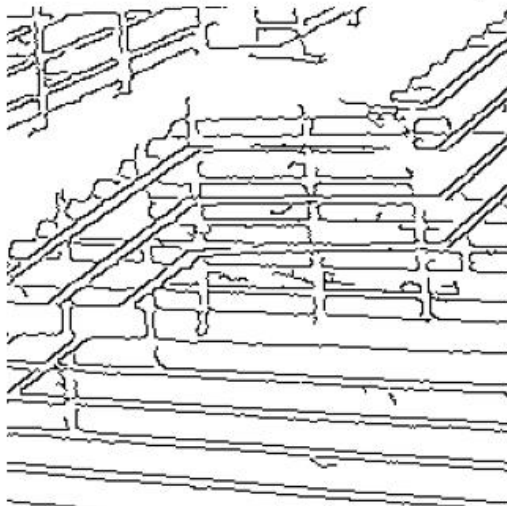
Разные визуальные эффекты и алгоритмы (очистка от шума, поиск контуров, повышение резкости и поиск объектов) под капотом используют **одну и ту же базовую математическую операцию свёртки скользящим окном**.

Canny по шумному кадру:
873586 краевых px



детектор границ Кэнни на зашумленном изображении без подготовки, он примет цифровой шум за реальные контуры

Гаусс $\sigma 1.5 \rightarrow$ Canny:
105302 px (чистый — 246863)



детектор границ Кэнни после сглаживающего фильтра Гаусса находит **физически обоснованные контуры** ступеней и перил

резкость: unsharp mask,
 $\alpha = 1, \sigma = 1.5$



С помощью свёртки специальным ядром (вычитающим размытую копию из оригинала) можно усилить контраст на границах

поиск шаблона при другом свете:
ZNCC = 0.99, рамка на месте

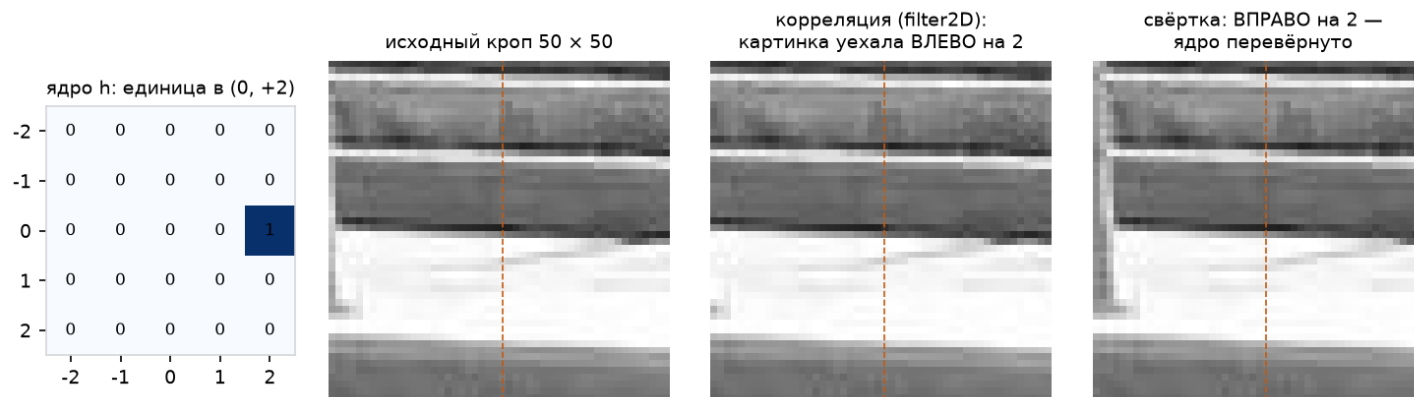


Использование метрики ZNCC позволяет детектировать объект на лестнице (оранжевая рамка) даже при сильном изменении условий освещения сцены.

Четыре применения одного оператора: сглаживание перед детектором краёв (874 тыс. ложных краевых пикселей $\rightarrow$ 105 тыс.), резкость через unsharp mask, поиск шаблона при другом освещении (ZNCC 0.99) — и всё это одна операция с разными ядрами

© фото — здание Университета Иннополис; собственная иллюстрация

- **Корреляция** (correlation): $g = \sum h(i, j) \cdot I(x + i, y + j)$ — ядро как есть; так работает `cv2.filter2D`
- **Свёртка** (convolution): $g = \sum h(i, j) \cdot I(x - i, y - j)$ — ядро перевёрнуто по обеим осям; так в учебниках и в названии «свёрточная сеть»
- Ядро «единица справа на 2»: корреляция **сдвигает кадр влево**, свёртка — вправо, расхождение **250 кодов**; после `cv2.flip(k, -1)` — **0**; для симметричных ядер — 0 всегда



`filter2D` — корреляция. Хотите свёртку — переверните ядро; для симметричных ядер это одно и то же

© фото — здание Университета Иннополис; собственная иллюстрация

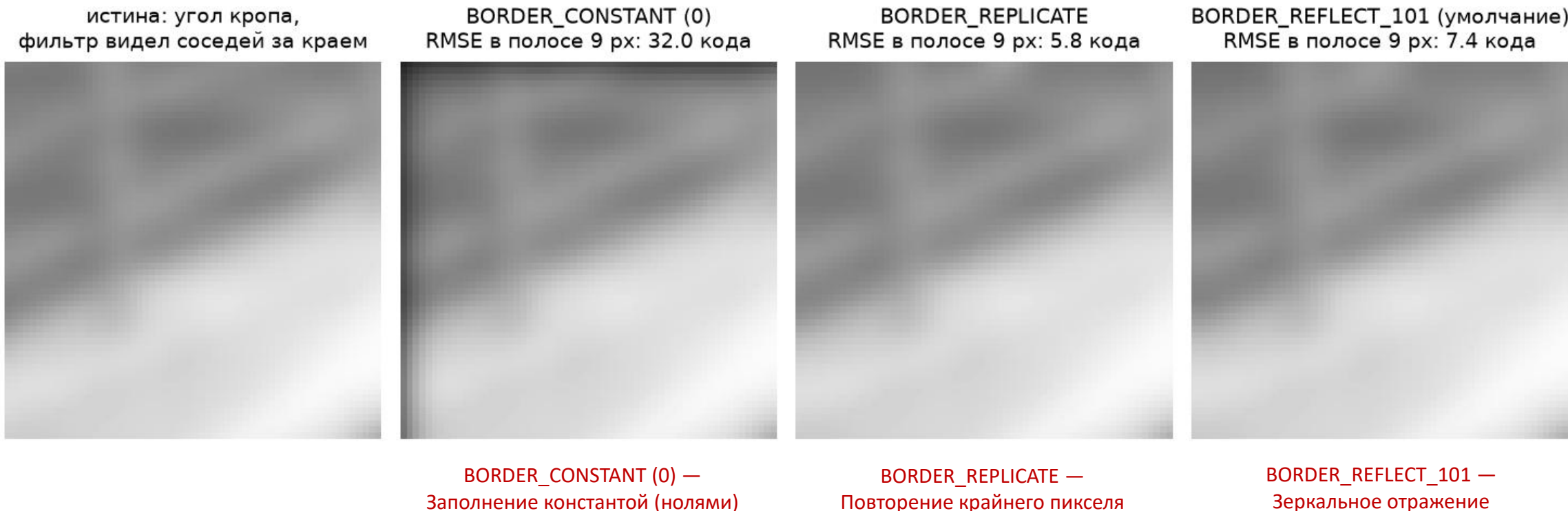
четыре фундаментальных математических свойства LSI-фильтров
(Linear Shift-Invariant / Линейных сдвигоинвариантных фильтров)

- **Линейность:** $h * (a \cdot I_1 + b \cdot I_2) = a \cdot (h * I_1) + b \cdot (h * I_2)$ — фильтр шумного кадра = фильтр чистого + фильтр шума (расхождение 0.0002 в float32)
- **Сдвиг-инвариантность** (shift invariance): сдвинуть и отфильтровать = отфильтровать и сдвинуть — **0 кодов** внутри кадра; ломается только у границы
- **Ассоциативность:** $h_2 * (h_1 * I) = (h_2 * h_1) * I$ — Гаусс σ 1.5, потом σ 2.0 = один Гаусс σ 2.5 ($\sqrt{1.5^2 + 2^2}$), расхождение 0.0
- **Коммутативность:** порядок двух линейных фильтров не важен; ЦПТ: box трижды $\approx$ Гаусс (расхождение весов 0.012)

Из линейности следует главное: фильтр не различает «сигнал» и «шум» — он одинаково сглаживает и то и другое

Границы: что фильтр видит за краем

Проблема краевых эффектов (Border Effects) при фильтрации картинок - как разные режимы достройки границ в OpenCV влияют на искажение пикселей у краев кадра

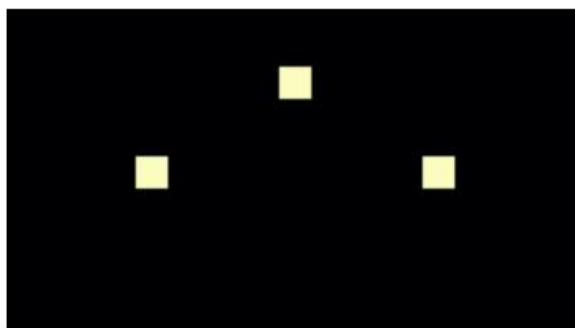


Кроп 200 × 200 с лестницы: «истина» — фильтр видел соседей за краем кропа. Три режима достройки: BORDER_CONSTANT даёт RMSE 32 кода в полосе 9 px — чёрная рамка; BORDER_REPLICATE — 5.8; BORDER_REFLECT_101 (умолчение OpenCV) — 7.4. Полоса ошибки — половина ядра

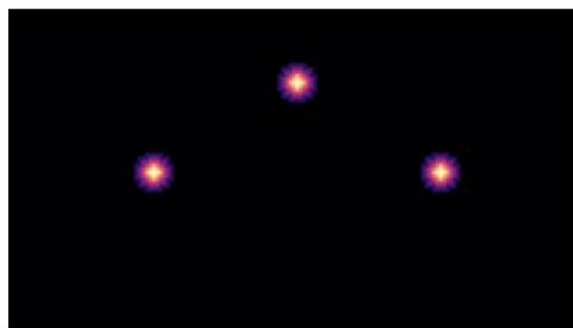
© фото — здание Университета Иннополис; собственная иллюстрация

Box-фильтр: просто, быстро и «звенит»

box 9×9 : точка $\rightarrow$ квадрат,
две точки $\rightarrow$ «решётка»



Гаусс $\sigma = 2.58$ (та же дисперсия):
точка $\rightarrow$ пятно



box 9 на шуме $\sigma 25$: 22.3 дБ



Гаусс $\sigma 2.58$: 23.1 дБ

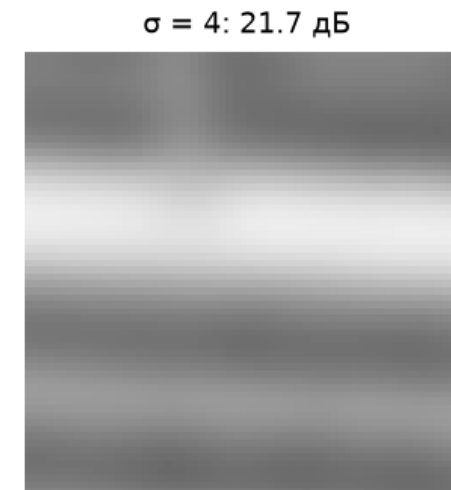
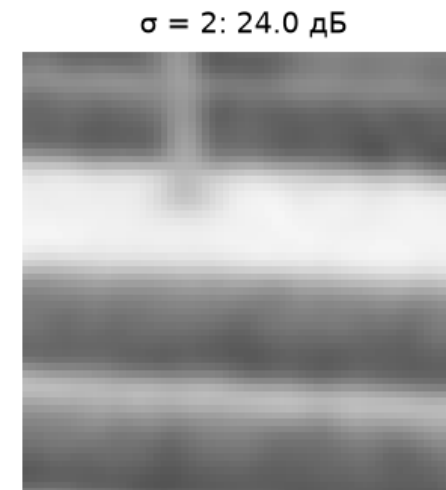
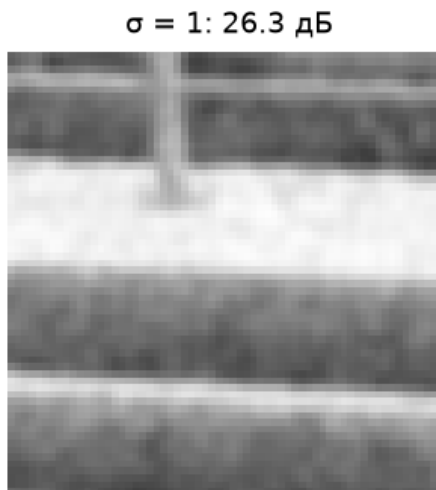
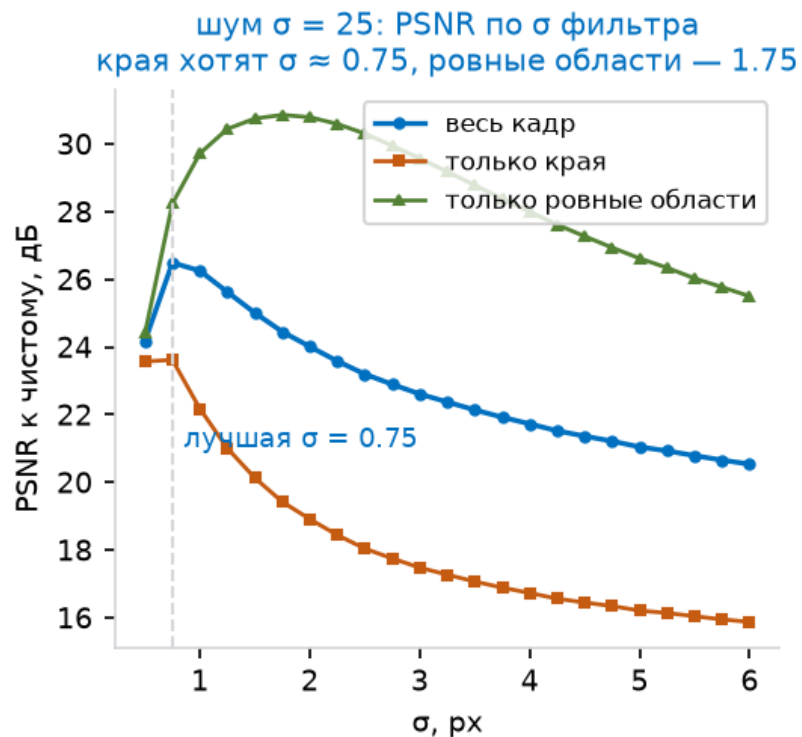


Хотя Box-фильтр (скользящее среднее) работает очень быстро, он **математически груб и создает неприятные визуальные артефакты**. Фильтр Гаусса при той же дисперсии справляется с задачей качественнее и бережнее

Box 9×9 превращает точку в квадрат, две точки — в решётку из наложений; Гаусс с той же дисперсией ($\sigma 2.58$) — в круглое пятно. На шуме $\sigma 25$: box 9 — 22.3 дБ, Гаусс $\sigma 2.58$ — 23.1: при равном подавлении шума box размывает больше и оставляет полосы вдоль ступеней

© фото — здание Университета Иннополис; собственная иллюстрация

Измерение: какая σ лучше



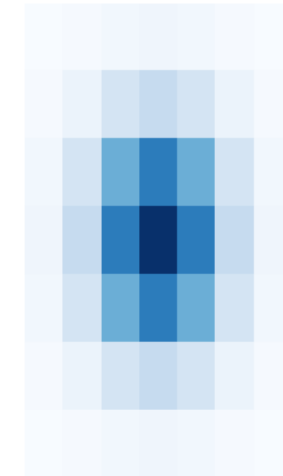
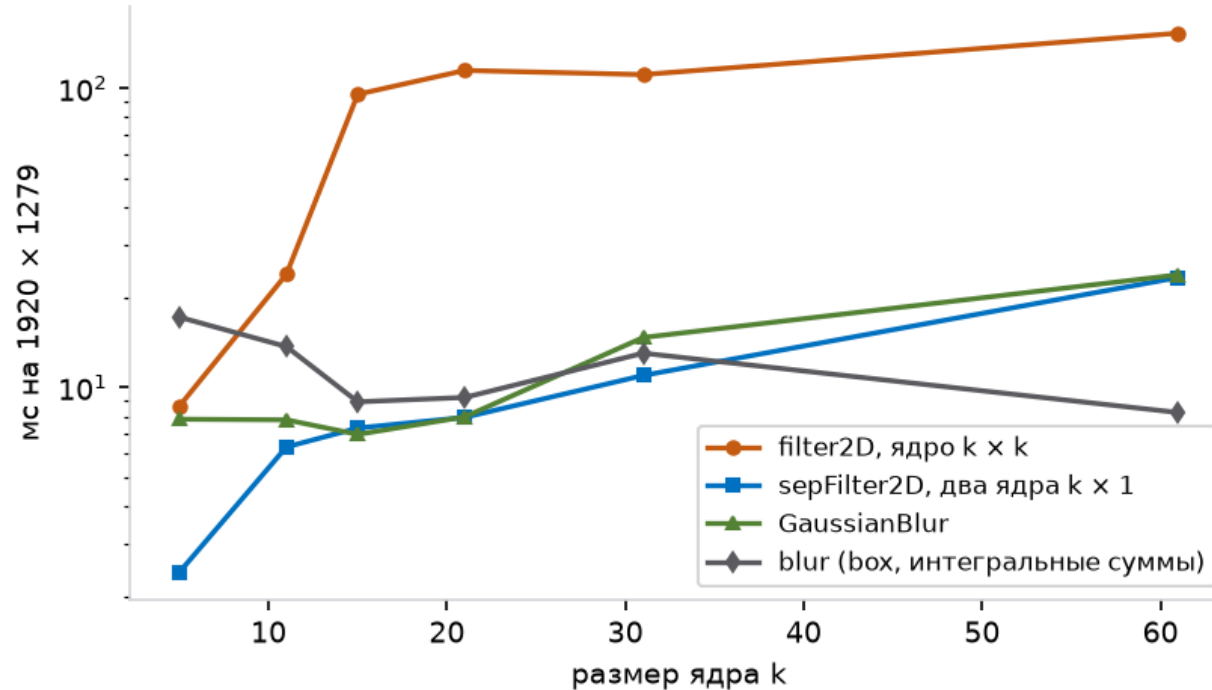
Компромисс при выборе параметра размытия σ (сигма) для подавления цифрового шума: Границы объектов (края) требуют слабой фильтрации ($\sigma = 0.75$) для сглаживания шума, в то время как ровные области требуют сильной фильтрации ($\sigma = 1.75$)

Шум $\sigma = 25$: PSNR по σ фильтра. Весь кадр — лучшая $\sigma 0.75$ (26.5 дБ), потом падает; края хотят $\sigma 0.75$, ровные области — 1.75. $\sigma 1$ — 26.3, $\sigma 2$ — 24.0, $\sigma 4$ — 21.7: правило « $\sigma \approx 1.5$ » из L4 было про антиалиасинг, а не про шум

© фото — здание Университета Иннополис; собственная иллюстрация

Сепарабельность: k^2 против $2k$

цена свёртки: полное ядро растёт как k^2 , сепарабельное — как $2k$, box — не растёт



сепарабельное ядро:
столбец × строка

31×31 : 961 умножений
на пиксель → 62

результат совпадает
до 1 кода

$$K = v \cdot v^T \quad (7 \times 7)$$

Гаусс сепарабелен: $K = v \cdot v^T$, два одномерных прохода вместо одного двумерного — для 31×31 это 62 умножения на пиксель вместо 961, результат совпадает до 1 кода. На 1920×1279 : filter2D с ядром 61 — 46 мс, sepFilter2D — 5.6, GaussianBlur — 9.7, а box любого размера — около 3 мс

Рост размера ядра размытия k может парализовать вычисления «в лоб», но сепарабельность и интегральные суммы решают эту проблему.

© собственная иллюстрация

Операция на 1920 × 1279, 8 бит	k = 5	k = 15	k = 31	k = 61
filter2D, ядро k × k	2.0 мс	30	27	46
sepFilter2D, два ядра k × 1	1.3	1.7	3.2	5.6
GaussianBlur	1.0	2.3	4.5	9.7
blur (box)	4.6	4.6	2.6	2.4
medianBlur 3 / 5 · bilateralFilter 9 · NLM	1.4 / 6 · 20–40 · 400–1100			

Тайминги плавают от машины к машине — на слайде порядок величины; все линейные фильтры укладываются в кадр при 30 FPS

Количественная таблицу таймингов (в миллисекундах) для различных методов фильтрации в OpenCV на изображении разрешения **FullHD (1920 × 1279, 8 бит)**. Она дает конкретные числовые ориентиры для разработчика, показывая порядки величин и применимость алгоритмов в реальном времени.

$I: 4 \times 4$

5	2	3	4
1	5	4	2
2	2	1	3
4	1	5	2

$S = \text{cv2.integral}(I)$: сумма всего
выше и левее (нулевой ряд добавлен)

0	0	0	0	0
0	5	7	10	14
0	6	13	20	26
0	8	17	25	34
0	12	22	35	46

сумма оранжевого окна 2×2 :

$$S[3,3] - S[1,3] - S[3,1] + S[1,1]$$
$$= 25 - 10 - 8 + 5 = 12 \quad (\text{прямой счёт: } 12)$$

четыре чтения — любое окно, любого размера

cv2.blur на 1920×1279 : $k = 5$ — 13.5 мс,
 $k = 101$ — 13.2 мс; cv2.integral — 13.1 мс

$S = \text{cv2.integral}(I)$: в каждой клетке — сумма всего выше и левее. Сумма любого окна — четыре чтения: $S[3,3] - S[1,3] - S[3,1] + S[1,1] = 25 - 10 - 8 + 5 = 12$, прямой счёт тоже 12. Поэтому cv2.blur на 1920×1279 стоит одинаково при $k = 5$ и $k = 101$ — около 3 мс, как и сам cv2.integral

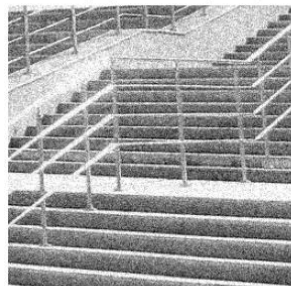
Чтобы посчитать сумму пикселей внутри любого прямоугольного окна, **не нужно перебирать и складывать каждый пиксель внутри него**. Достаточно один раз построить интегральную матрицу (где хранится сумма всех пикселей исходного кадра I , находящихся выше и левее неё), а затем вычислять сумму в любом окне всего за **4 обращения к памяти** (4 чтения) и 3 простейшие операции (сложение/вычитание).

© собственная иллюстрация

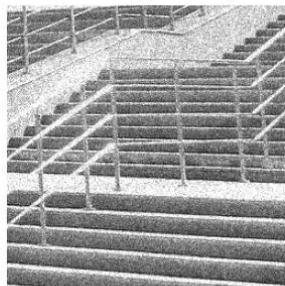
Модели шума: три разных врага

Три классические математические модели цифрового шума в компьютерном зрении

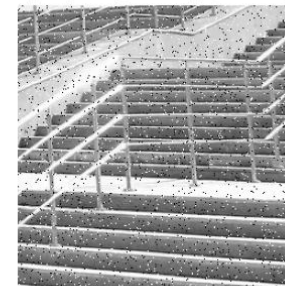
Гаусс $\sigma = 25$ (шум чтения, ISO): 20.7 дБ



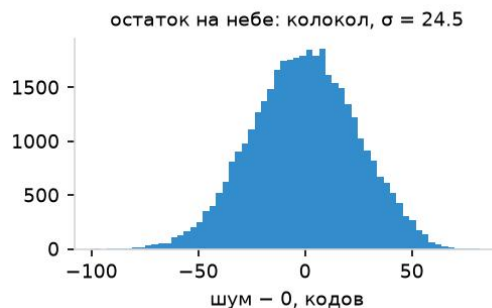
Пуассон, $\sigma^2 \propto I$ (дробовой): 21.6 дБ



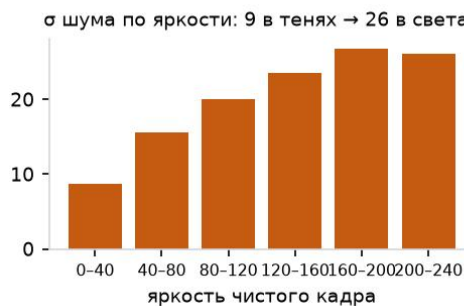
соль и перец 5 % (битые px, канал): 17.7 дБ



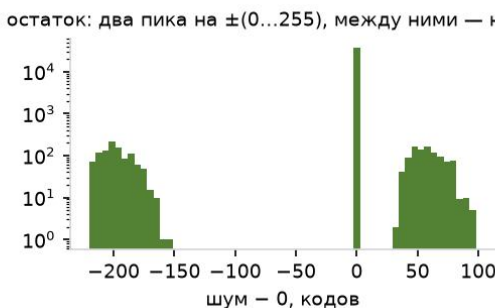
Гауссов шум (Аддитивный / Шум чтения)



Пуассоновский шум (Дробовой / Квантовый)



Импульсный шум («Соль и перец»)



Гаусс $\sigma = 25$ — шум чтения и усиления (ISO): остаток — колокол с $\sigma = 24.5$, не зависит от яркости. Пуассон — дробовой шум: $\sigma^2 \propto I$, в тенях 9 кодов, в светах 26 — тот же закон, что кривая шума сенсора в L2. Соль и перец 5 % — битые пиксели и потери в канале: остаток — два пика на ± 255 , между ними ничего

© фото — здание Университета Иннополис; собственная иллюстрация

Шум × фильтр → PSNR, дБ

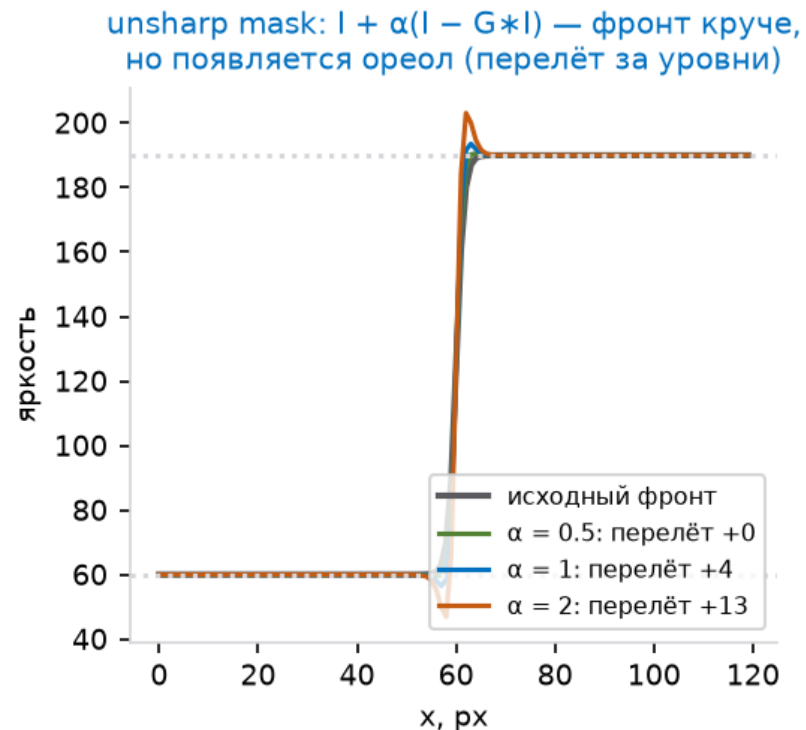
Шум → фильтр	без	box 5	Гаусс 1.5	медиана 5	билатеральный	NLM
Гаусс σ 25	20.7	24.1	25.0	24.8	26.5	28.0
Пуассон	21.6	24.1	25.0	24.9	26.5	28.0
соль-перец 5 %	17.7	23.7	24.5	26.1	20.6	23.0
время, мс	0	3	2	6	20–40	400–1100

Та же таблица — TODO 3 занятия 5: строки и столбцы вы заполните сами и добавите свои фильтры

Итоговая сравнительная матрица эффективности разных фильтров против разных типов шума, а также их «вычислительную стоимость» (время работы в миллисекундах)

Под гауссов и дробовой шум — билатеральный или NLM; под соль-перец — только медиана; box проигрывает Гауссу везде

Резкость: unsharp mask



$I + \alpha(I - G*I)$: вычитаем размытую копию и добавляем разницу. Фронт становится круче, но появляется ореол — перелёт за уровни: $\alpha 0.5$ — +0.5 кода, $\alpha 1$ — +3.5, $\alpha 2$ — +13. Резкость усиливает всё, что не сигнал: JPEG q50 с 35.0 дБ падает до 26.6, шум $\sigma 10$ — с 28.4 до 21.8

Чтобы сделать картинку визуально более резкой, алгоритм **выделяет мелкие детали (высокие частоты) и искусственно усиливает их контраст**. Для этого из исходного изображения вычитают его размытую копию, а полученную разницу (карту контуров) добавляют обратно к оригиналу с коэффициентом

© фото — здание Университета Иннополис; собственная иллюстрация

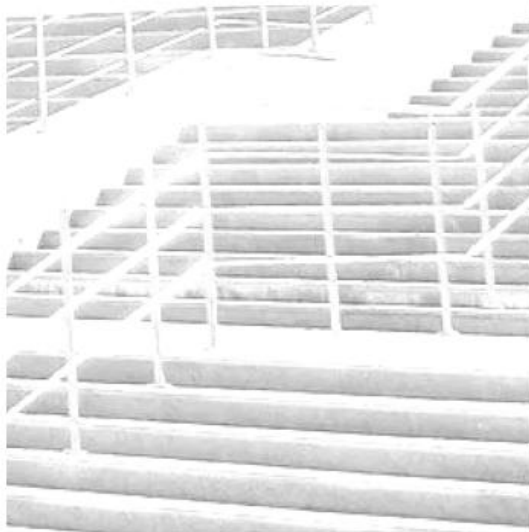
`uint8` переполняется молча

Опасность незаметного переполнения типов данных (uint8) при арифметических операциях и фильтрации изображений в коде (например, в NumPy и OpenCV)

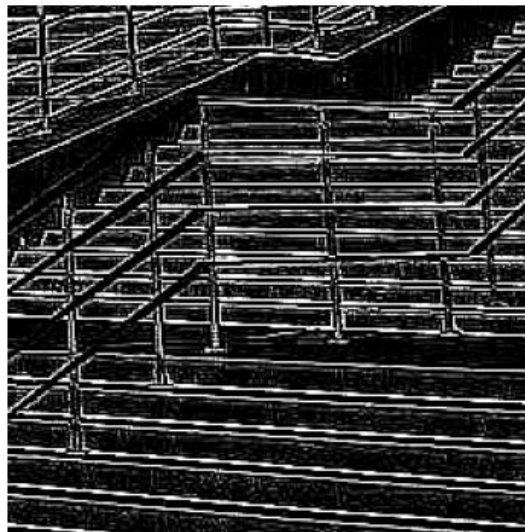
G + 100 в uint8: 42 % пикселей
«перевернулись» ($200 + 100 = 44$)



то же через int и np.clip:
светлое остаётся светлым



Laplacian, ddepth = CV_8U:
отрицательные 44 % стали нулём



Laplacian, CV_32F:
знак сохранён (вход - / выход +)



G + 100 в uint8: 42 % пикселей «перевернулись» — $200 + 100 = 44$, светлое стало чёрным; через int и np.clip светлое остаётся светлым. Laplacian с ddepth = CV_8U: 44 % отрицательных откликов стали нулём, чёрных пикселей 56 % против 12 % во float32 — половина информации о знаке потеряна

Стандартный тип данных для картинок — это uint8 (целые числа от 0 до 255). Если в результате сложения или вычисления производных значение выходит за эти рамки, **происходит автоматическое зацикливание (переполнение) или обрезка**, что полностью уничтожает картинку и ломает логику алгоритмов.

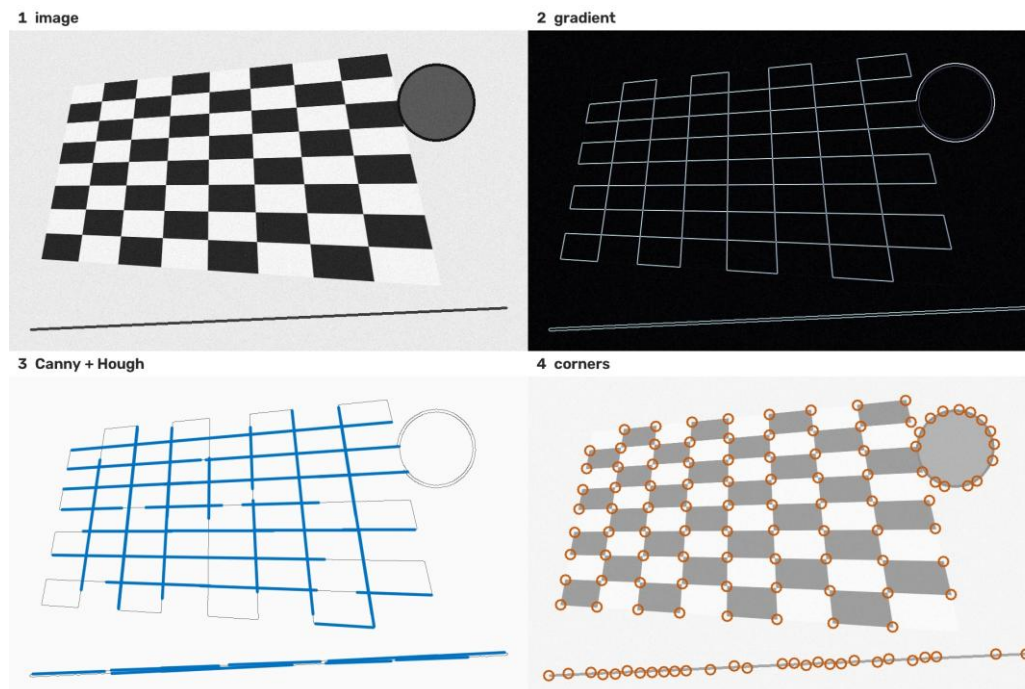
© фото — здание Университета Иннополис; собственная иллюстрация

- **L2, демозаик:** билинейная интерполяция мозаики Байера — это свёртка с ядром $\begin{bmatrix} 1 & 2 \\ 1 \end{bmatrix}^T \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} / 4$ по маске канала; **стек 16 кадров** делил σ на 4 — сегодня то же делает окно
- **L3:** адаптивный порог — локальное среднее по окну (box + интегральное изображение); CLANE — нелинейная операция над окрестностью, как медиана
- **L4:** GaussianBlur $\sigma = 1.5$ перед уменьшением $\times 1/4$ — антиалиасинг (27.9 $\rightarrow$ 36.7 дБ); INTER_AREA — box-фильтр плюс прореживание
- **Дальше:** L6 объяснит фильтры через спектр — почему box «звенит»; L7 — производные как ядра, Собель, Canny; L8 — матрица вторых моментов из сглаженных градиентов

Свёртка — сквозной инструмент курса: пять лекций назад мы её уже использовали, не называя

- **Свёрточный слой** (convolution layer) — сегодняшний `filter2D` с ядром, которое учится; первый слой обученной сети — Гауссы, производные и цветные пятна, как в 2012 году у AlexNet
- **Шумоподавление:** BM3D (2007) — «NLM плюс спектр»; DnCNN и современные модели учат остаток I — чистое и выигрывают у NLM 2–4 дБ, но требуют примеры шума данной камеры
- **Где классика остаётся:** предобработка на борту (2 мс против сотен), аугментации, детерминированность и объяснимость — фильтр можно проверить формулой $\sigma_{out} = \sigma \cdot \sqrt{\sum h^2}$
- **Мост:** билатеральный фильтр — прямой предок attention: вес соседа зависит от его похожести

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru

Демо-код лекции: [lectures/L05-filtering/demo/](#)